

An aerial photograph of the European Spallation Source (ESS) facility, featuring modern buildings, a large circular structure, and a row of flags. The image is overlaid with a semi-transparent blue filter. A white logo consisting of the letters 'ess' inside a circle with two intersecting arcs is positioned on the left side.

ess

**EUROPEAN
SPALLATION
SOURCE**



Machine Learning in EPICS

Olle Settergren, Joel Utsi, Mohammadreza Zamani

Project overview



1. Feed-Forward Neural Network:

Classify one scalar input as LOW/MID/HIGH using EPICS records.

2. Recurrent Neural Network:

Add memory of the previous hidden state to classify FALLING/STABLE/RISING.

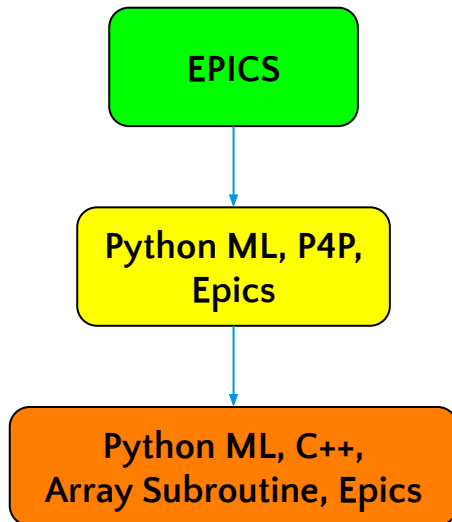
3. Time-Series Forecasting:

Train an AutoGluon model to enable forecasting for 48 hours through EPICS PVs and visualize.

3. Leak Location Classifier:

Train an mlpack DecisionTree model and run it from C++ by means of an aSub record.

Learning Progression





Feed-Forward Neural Network



Feed-forward Architecture

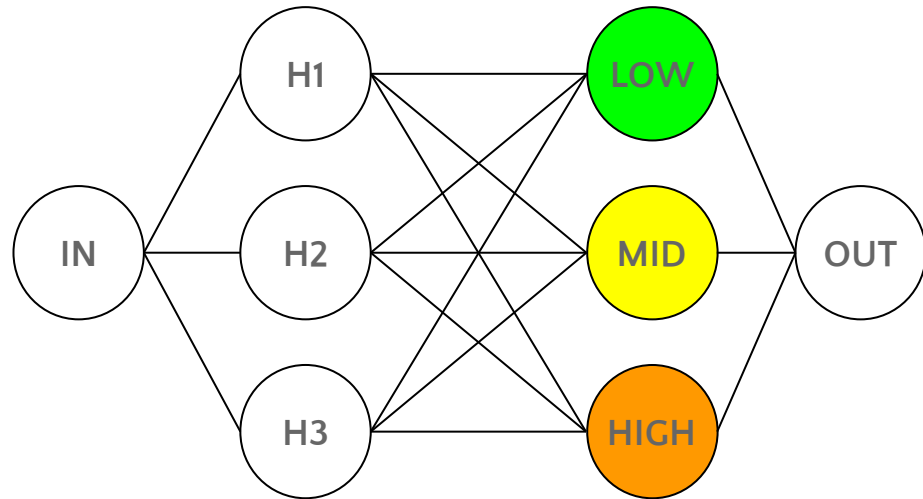
Input: 0-100, normalized to [0.0, 1.0]

Hidden Layer

Output: Neuron activations are converted into classes

- 0-34: LOW
- 35-65: MID
- 66-100: HIGH

Final Class: **LOW**, **MID**, and **HIGH**.



Input Layer

Hidden Layer

Output layer

Implementation

Two different approaches



- Each neuron needs 1-2 calc steps -> Lots of records.
- Calculations are similar -> Generalize?

Solution 1 (works for small networks)

- Hardcode all neurons in a .db file

Solution 2 (more scalable)

- Define .template files for neurons, weights, etc.
- .substitutions file generates records in bulk from .templates and params

(Simplified) code example



neuron.template

```
record(calcout, "$(P)$(R)$(node)") {  
  field(INPA, "$(P)$(R)$(bias)")  
    field(INPB, "$(P)$(R)$(in1)")  
    field(INPC, "$(P)$(R)$(w1)")  
  
  ...  
  field(CALC, "A+B*C+D*E+...")  
  field(FLNK, "$(P)$(R)$(next_node)")  
}
```

weights.template

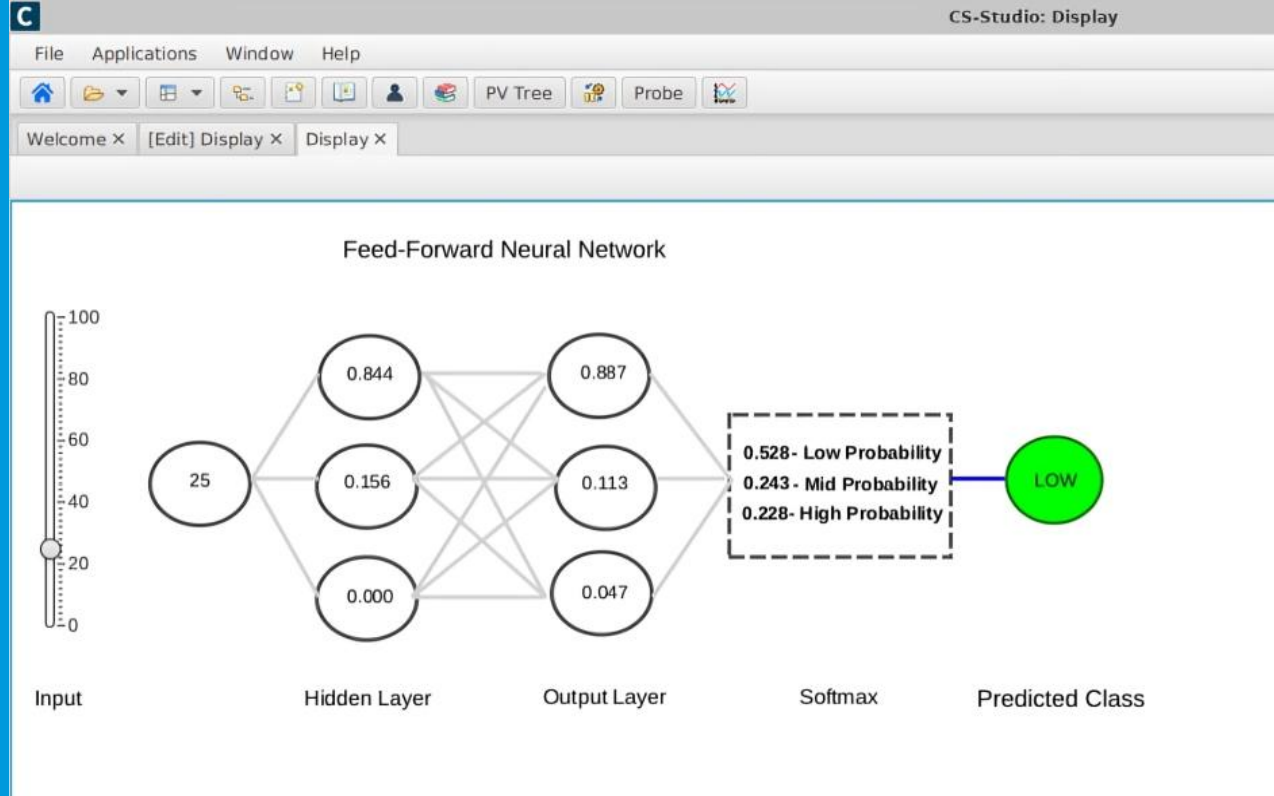
```
record(ai, "$(P)$(R)W_$(in)_$(out)") {  
  field(VAL, "$(weight)")  
  field(PINI, "YES")  
}
```

network.substitutions

```
file neuron.template {  
  pattern {node, next_node, bias, in1, w1, ... }  
    {H1, H2, B_H1, IN, W_IN_H1, ... }  
    {H2, H3, B_H2, IN, W_IN_H2, ... }  
  
  ...  
}  
  
file weights.template {  
  pattern {in, out, weight}  
    {IN, H1, -20}  
    {IN, H2, 10}  
  
  ...  
}
```

... repeat for activation functions, softmax etc.

Phoebus Result





Recurrent Neural Network

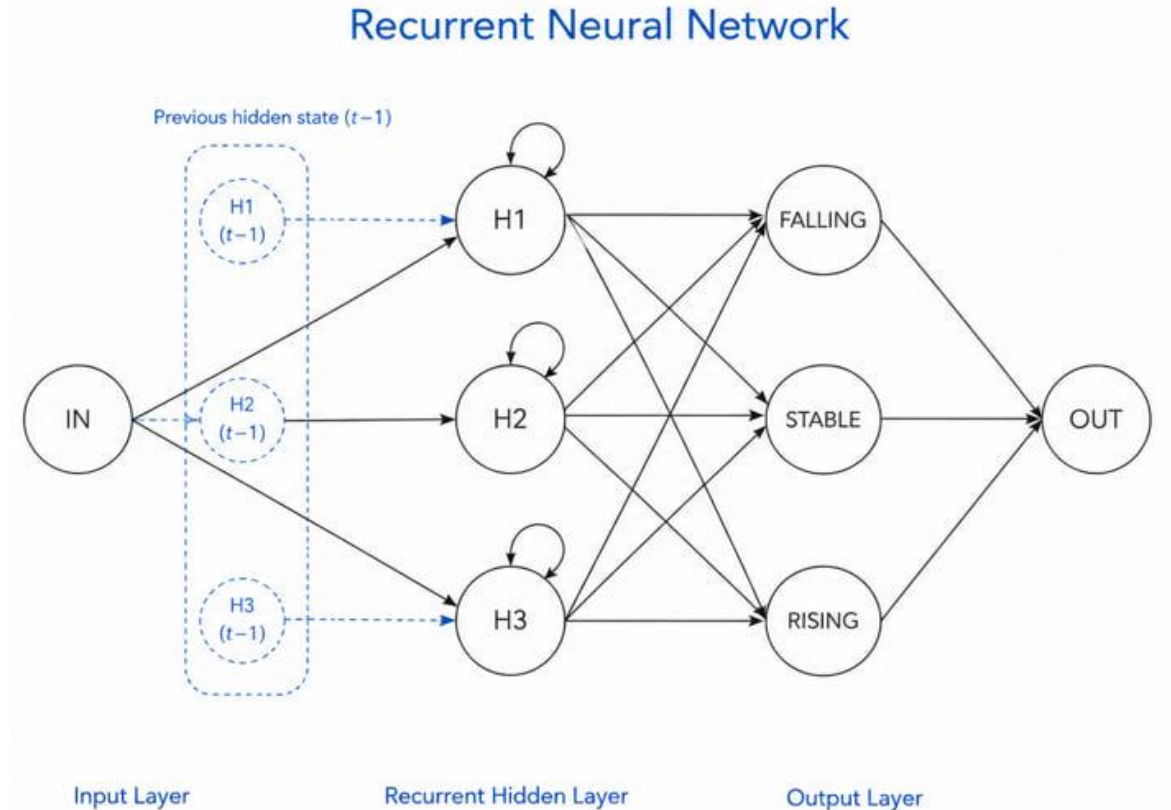
Input: 0-100, normalized to [0.0, 1.0]

Hidden Layer: Each neuron has inputs

IN: and the previous hidden neuron states.

Output: Neuron activations are converted into classes:

FALLING, STABLE, and RISING.



Phoebus Result

STABLE

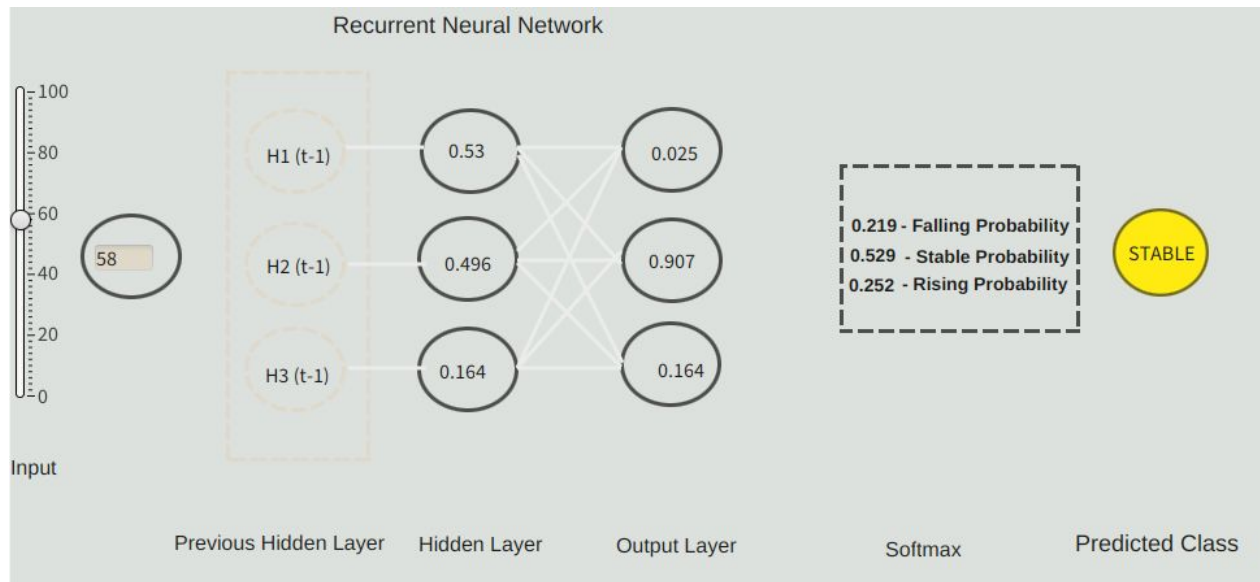
57 → 58

RISING

58 → 70

FALLING

70 → 40



Phoebus Result

STABLE

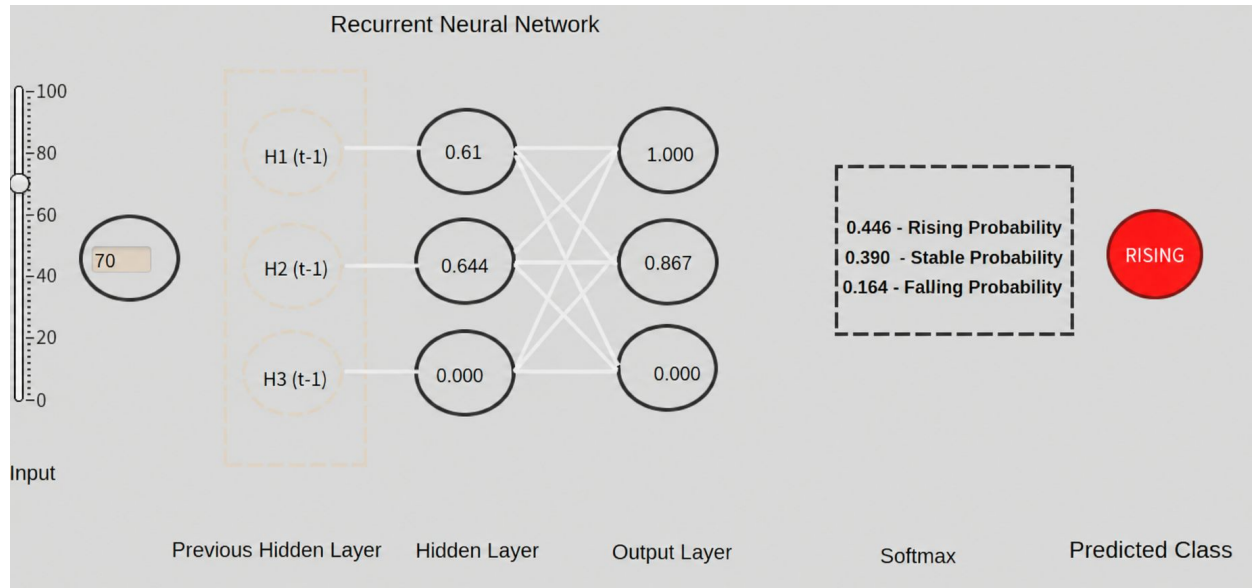
57 → 58

RISING

58 → 70

FALLING

70 → 40



Phoebus Result

STABLE

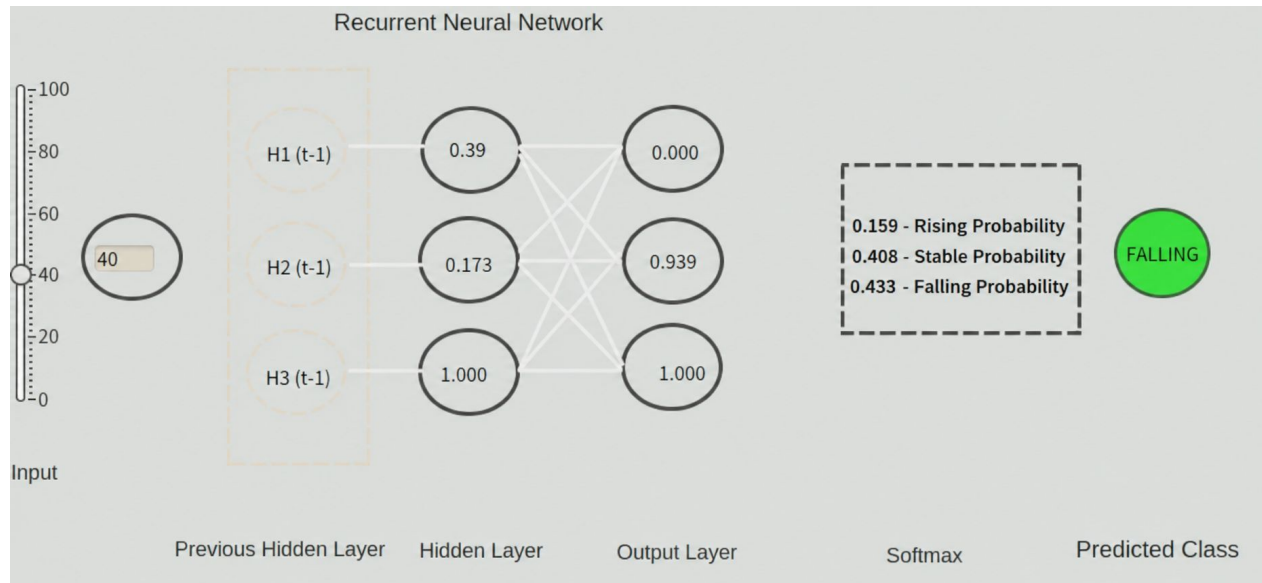
57 → 58

RISING

58 → 70

FALLING

70 → 40





Time-Series Forecasting

Time-Series Forecasting

Train offline in Python, save the predictor, and then expose forecasts through EPICS.

Forecast Configuration:

AutoGluon trains multiple models and chooses the best model automatically given prediction length and evaluation metric.

Data Split:

April Data: 698 hourly samples for training

May Data: 722 hourly samples for testing

Model	Static features (continuous + categorical)	Known covariates (continuous + categorical)	Past covariates (continuous + categorical)
<code>DirectTabularModel</code>	✓	✓	
<code>RecursiveTabularModel</code>	✓	✓	
<code>DeepARModel</code>	✓	✓	
<code>PatchTSTModel</code>		✓	
<code>TemporalFusionTransformerModel</code>	✓	✓	✓
<code>TiDEModel</code>	✓	✓	
<code>WaveNetModel</code>	✓	✓	
<code>Chronos2Model</code>		✓	✓

Some of the models utilized by AutoGluon

Forecast Result

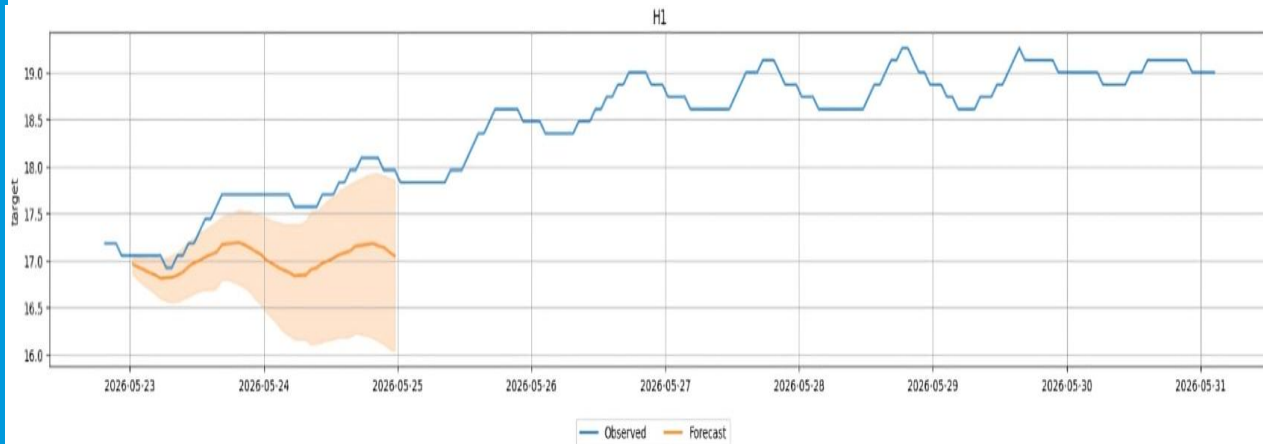
48-hour mean forecast with
10-90% quantile uncertainty band.

Output:

Mean forecast

High (0.9q) forecast

Low (0.1q) forecast



48-hour horizon:

The predictor outputs one mean value for each future hour.

Uncertainty:

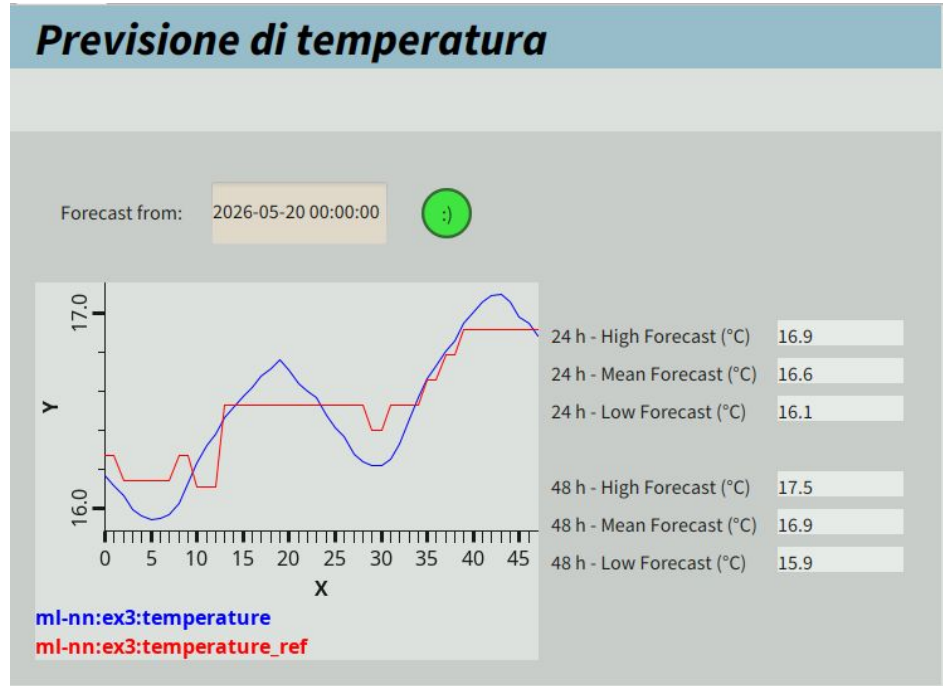
Quantiles 0.1 and 0.9 define a practical forecast interval.

OPI

Implementation in Phoebus OPI.

Forecasting ESS temperature using pre-trained model on data for 2 days into the future.

2026-05-20 00:00:00 -> 2026-05-22 00:00:00

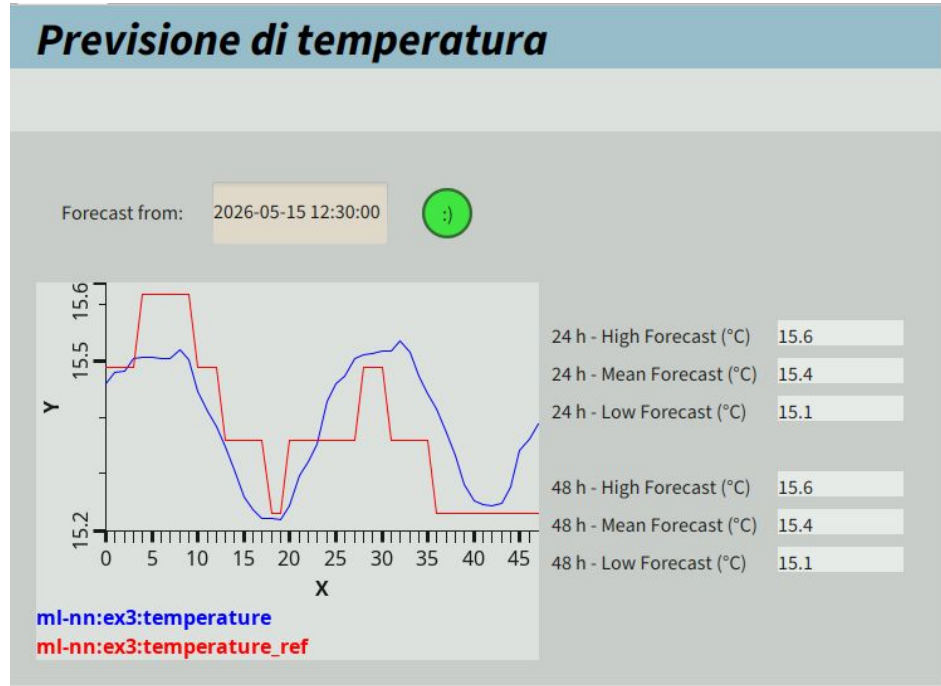


OPI

Implementation in Phoebus OPI.

Forecasting ESS temperature using pre-trained model on data for 2 days into the future.

2026-05-15 12:30:00 -> 2026-05-17 12:30:00

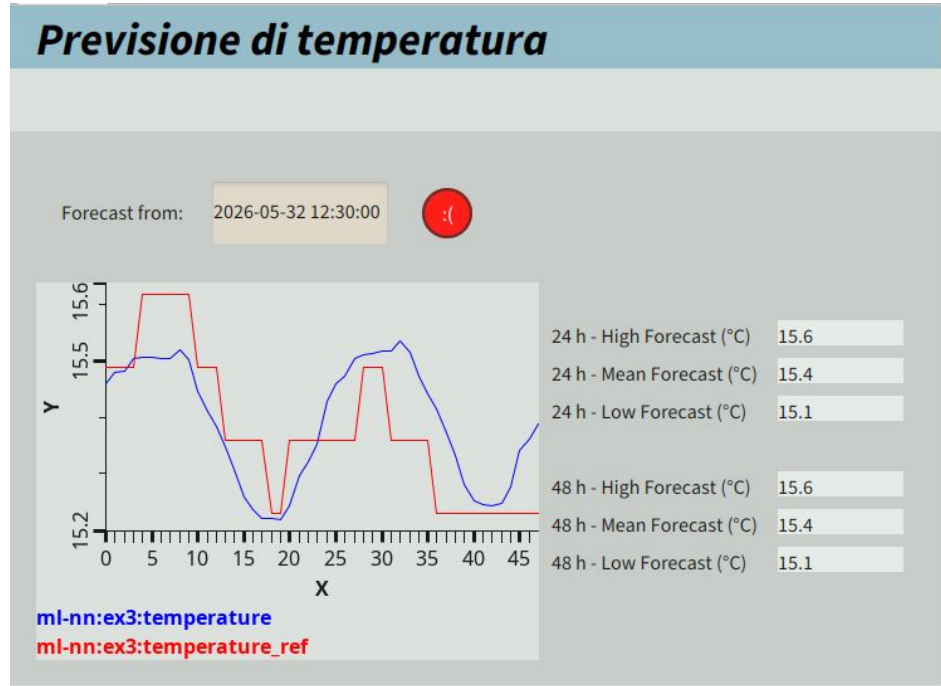


OPI

Implementation in Phoebus OPI.

Forecasting ESS temperature using pre-trained model on data for 2 days into the future.

2026-05-32 12:30:00 -> 2026-05-34 12:30:00





Leak-Location Classifier

Python:

Used to train a Decision Tree model with mlpack. The trained parameters were saved in a binary .bin file.

70/10/20 - Train/Val/Test

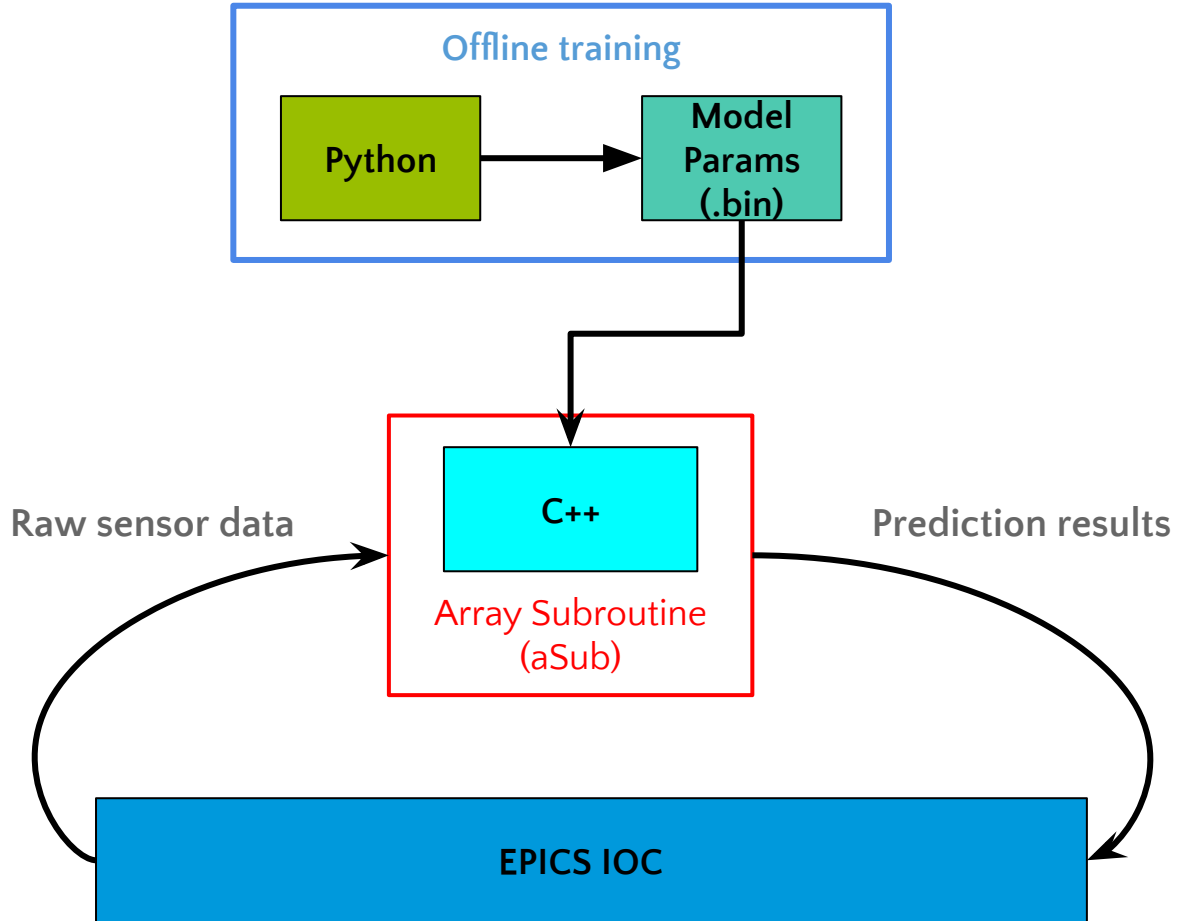
C++:

C++ acts a glue inside of EPICS using array subroutine (**aSub**) records.

The trained Python model parameters are imported as a .bin binary file and used for prediction.

EPICS:

Records for sensor input and displaying results
(Leak Location and Confidence)



Horrors of C++

Reading from and writing to EPICS



```
double tt102    = *((epicsFloat64*) prec->a);  
double tt110    = *((epicsFloat64*) prec->b);  
double li103    = *((epicsFloat64*) prec->c);  
double pspeed = *((epicsFloat64*) prec->d);
```

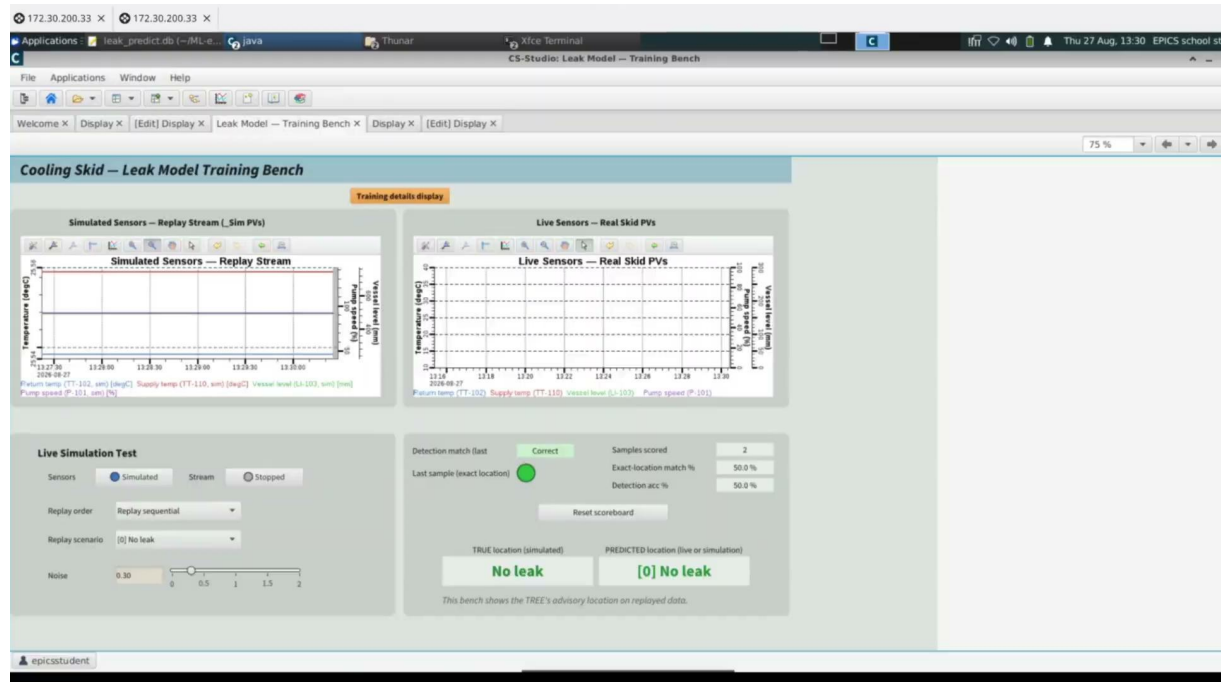
```
*(double*) prec->vala = static_cast<double>(prediction);  
*(double*) prec->valb = conf;
```

... repeat for activation functions, softmax etc.

OPI

Implementation in Phoebe OPI.

Leak-Location Predictions





EUROPEAN
SPALLATION
SOURCE

Thank you!

Questions?



ess.eu



[@ess.neutron](https://www.instagram.com/ess.neutron)



[@ess-neutron](https://www.linkedin.com/company/ess-neutron)



[@essneutron](https://www.youtube.com/channel/UCessneutron)



EUROPEAN
SPALLATION
SOURCE



EUROPEAN
SPALLATION
SOURCE

Follow the ESS Road to Science



ess.eu



@ess.neutron



@ess-neutron



@essneutron